

Clients



HttpClient + Probleme bei der Verwendung

Wichtig: HttpClient-Klasse kann problemlos verwendet werden ☺

Problem: Die meisten Entwickler verwenden HttpClient Klasse falsch:

- IDisposable wird nicht am Ende eines *using* aufgerufen.

Problem: Beim Verwerfen des HttpClient-Objekts, wird der zugrunde liegende Socket nicht sofort freigegeben

Folgen:

- Instanziieren einer HttpClient-Klasse für **jede Anforderung erschöpft die Anzahl der verfügbaren Sockets** und führt zu **hoher Auslastung**.
- Dieses Problem führt zu **SocketException-Fehlern**.

HttpClient + Probleme bei der Verwendung

Lösung:

- HttpClient wird nur einmal instanziiert
- In .NET Core 2.1 wurde die IHttpClientFactory-Schnittstelle eingeführt -> startup.cs -> AddHttpClient()

Weiterer Artikel->[Tutorial: Übermitteln von HTTP-Anforderungen in einer .NET-Konsolen-App mit C# | Microsoft Docs](#)

Vorteile der Verwendung von IHttpConnectionFactory (und IHttpMessageHandlerFactory)

- IHttpConnectionFactory (DefaultHttpClientFactory) ist eng mit NuGet-Paket Microsoft.Extensions.DependencyInjection gebunden
- Zentraler Ort (startup.cs) für das Benennen und Konfigurieren von HttpClient-Objekte.
- Unterstützt und Erweitert die Polly Library

4

Zentraler Ort für das Benennen und Konfigurieren von HttpClient-Objekte. -> Use Case -> Sie können beispielsweise einen Client (Dienst-Agent) konfigurieren, der für das Zugreifen auf einen bestimmten Microservice vorkonfiguriert ist.

Polly Library:

[Polly](https://pollyproject.azurewebsites.net/) ist eine Bibliothek zur Behandlung vorübergehender Fehler, mit der Entwickler ihren Anwendungen Resilienz hinzufügen können, indem sie einige vordefinierte Richtlinien auf fließende und threadsichere Weise verwenden. Link: <https://thepollyproject.azurewebsites.net/>

Mehrere Verwendungsmöglichkeiten für IHttpConnectionFactory

Es gibt mehrere Methoden, um IHttpConnectionFactory in Ihrer Anwendung zu verwenden:

- Grundlegende Verwendung
- Verwenden Sie benannte Clients.
- Verwenden Sie typisierte Clients.
- Verwenden Sie generierte Clients.

Verwendung typisierte Clients mit IHttpConnectionFactory

Default:

```
// Startup.cs
//Add http client services at ConfigureServices(IServiceCollection services)
services.AddHttpClient<ICatalogService, CatalogService>();
services.AddHttpClient<IBasketService, BasketService>();
services.AddHttpClient<IOrderingService, OrderingService>();
```

Mit Konfiguration:

```
services.AddHttpClient<ICatalogService, CatalogService>(client =>
{
    client.BaseAddress = new Uri(Configuration["BaseUri"]);
})
.AddPolicyHandler(GetRetryPolicy())
.AddPolicyHandler(GetCircuitBreakerPolicy());

static IAsyncPolicy<HttpResponseMessage> GetRetryPolicy()
{
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .OrResult(msg => msg.StatusCode == System.Net.HttpStatusCode.NotFound)
        .WaitAndRetryAsync(6, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));
}
```

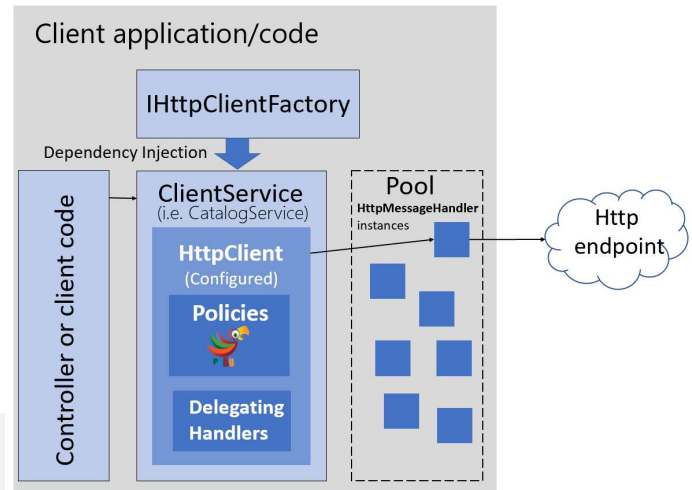


Abbildung: typisierte Clients mit IHttpConnectionFactory

Verwendung typisierte Clients mit IHttpClientFactory

- HttpClient wird via Konstruktor-Injection hinzugefügt
- Ein typisierter Client ist ein temporäres Objekt.
 - Eine neue Instanz wird erstellt, wenn eine benötigt wird.
- Die HttpResponseMessage-Objekte im Pool werden von mehreren HttpClientS wiederverwendet

```
public class CatalogService : ICatalogService
{
    private readonly HttpClient _httpClient;
    private readonly string _remoteServiceBaseUrl;

    public CatalogService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<Catalog> GetCatalogItems(int page, int take,
                                                int? brand, int? type)
    {
        var uri = API.Catalog.GetAllCatalogItems(_remoteServiceBaseUrl,
                                                    page, take, brand, type);

        var responseString = await _httpClient.GetStringAsync(uri);

        var catalog = JsonConvert.DeserializeObject<Catalog>(responseString);
        return catalog;
    }
}
```

Verwenden von typisierten Clientklassen

- Client-Klassen können Sie sie überall dort verwenden, wo Dienste mithilfe von DI eingefügt werden.

```
namespace Microsoft.eShopOnContainers.WebMVC.Controllers
{
    public class CatalogController : Controller
    {
        private ICatalogService _catalogSvc;

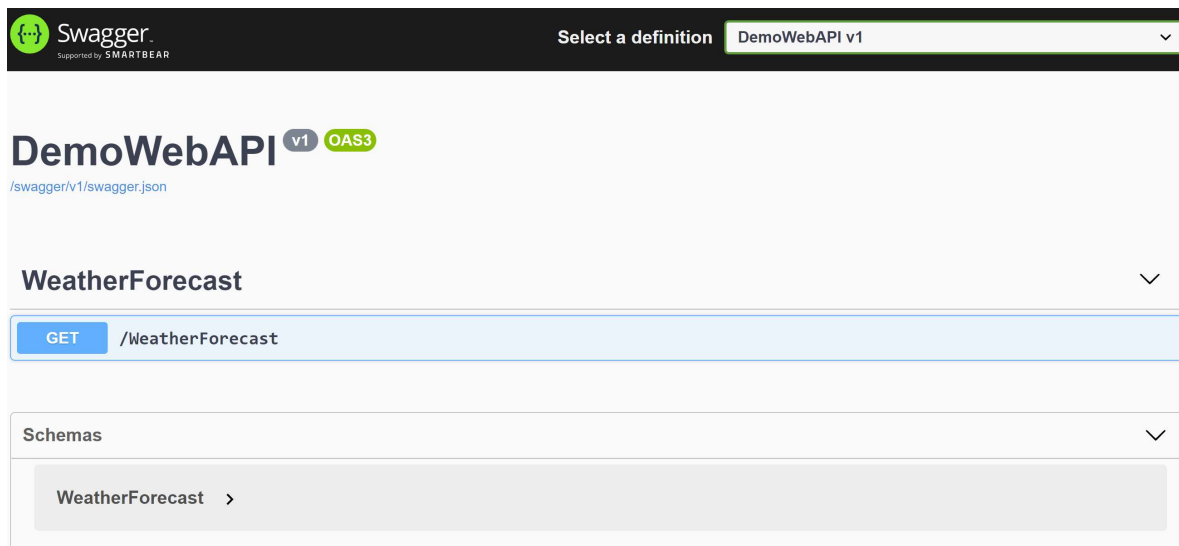
        public CatalogController(ICatalogService catalogSvc) =>
            _catalogSvc = catalogSvc;

        public async Task<IActionResult> Index(int? BrandFilterApplied,
            int? TypesFilterApplied,
            int? page,
            [FromQuery]string errorMsg)
        {
            var itemsPage = 10;
            var catalog = await _catalogSvc.GetCatalogItems(page ?? 0,
                itemsPage,
                BrandFilterApplied,
                TypesFilterApplied);

            //... Additional code
        }
    }
}
```

SwaggerClient Generierung

- Bei jedem Build-Vorgang wird die swagger.json neu erstellt
- Swagger.json beinhaltet eine Meta-Beschreibung der WebAPI



Swagger. Supported by SMARTBEAR

Select a definition DemoWebAPI v1

DemoWebAPI v1 OAS3

</swagger/v1/swagger.json>

WeatherForecast ✓

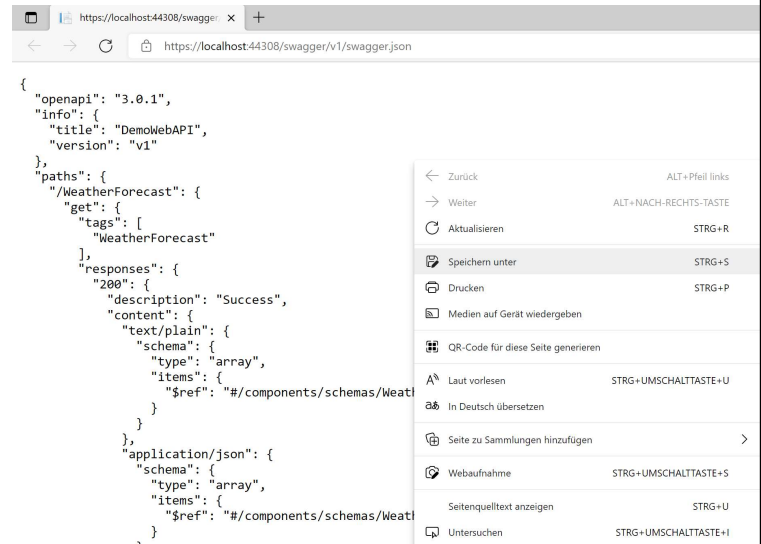
GET /WeatherForecast

Schemas ✓

WeatherForecast >

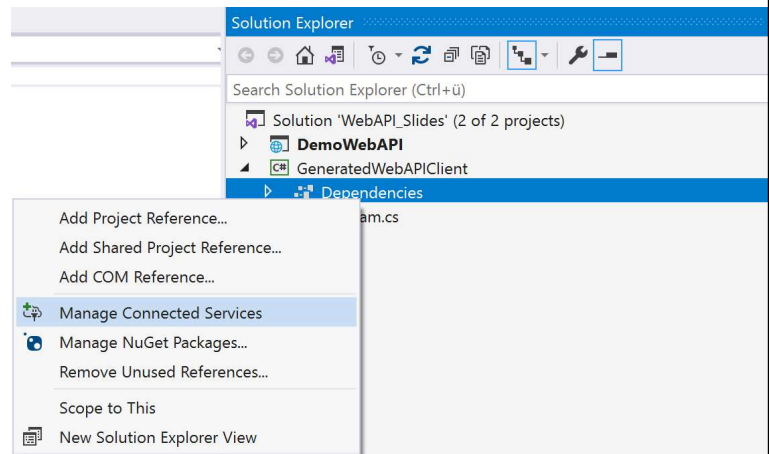
SwaggerClient Generierung

- Swagger.json speichern wir in unserem WebAPI-Projektverzeichnis ab.



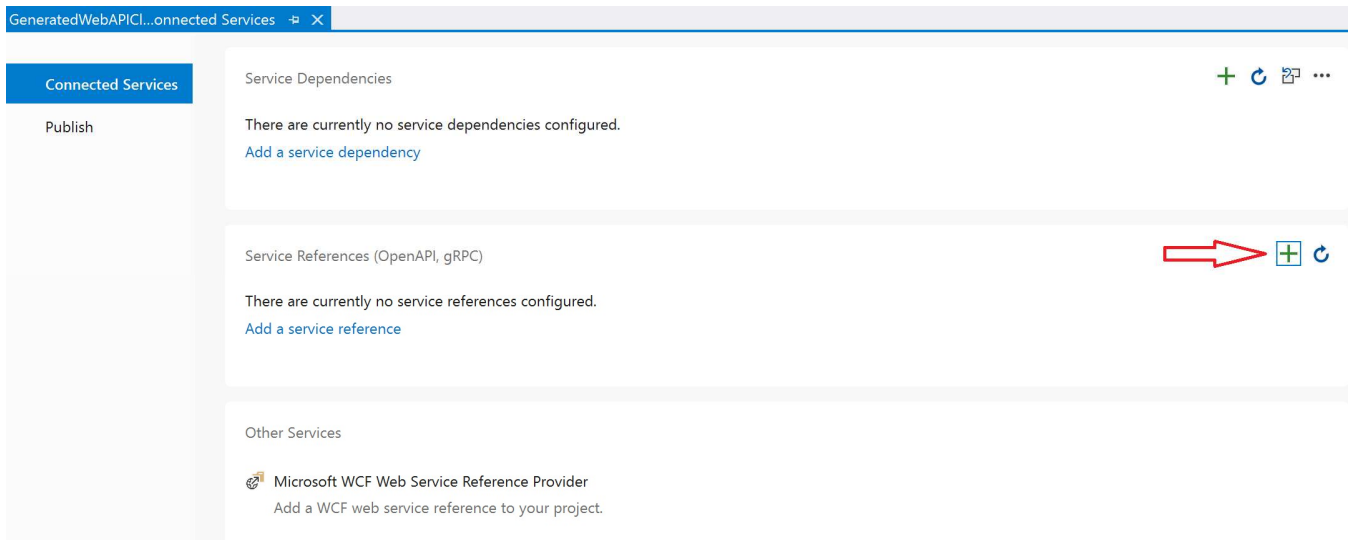
SwaggerClient Generierung

- Erstellen eine Konsolenanwendung für den Client
- Rechtsklick auf Dependencies -> Manage Connected Services



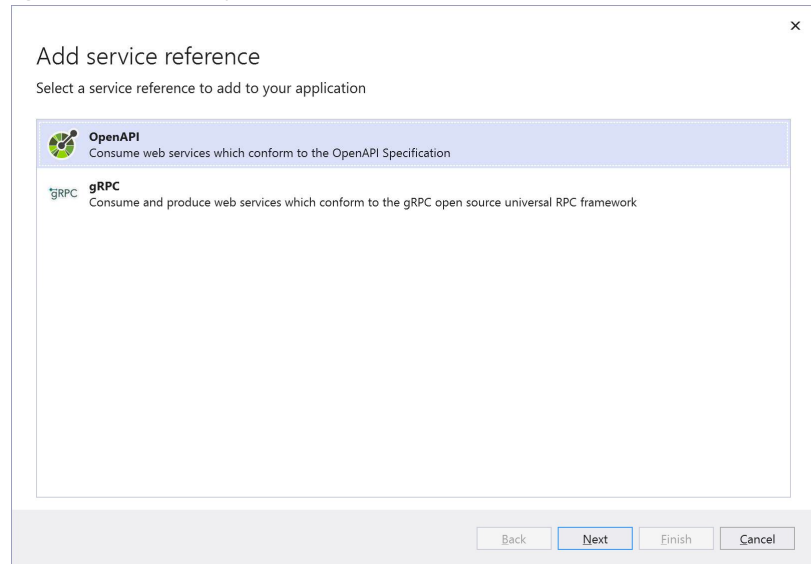
SwaggerClient Generierung

- Klicke auf „Add a service reference“ oder auf + (siehe roter Pfeil)



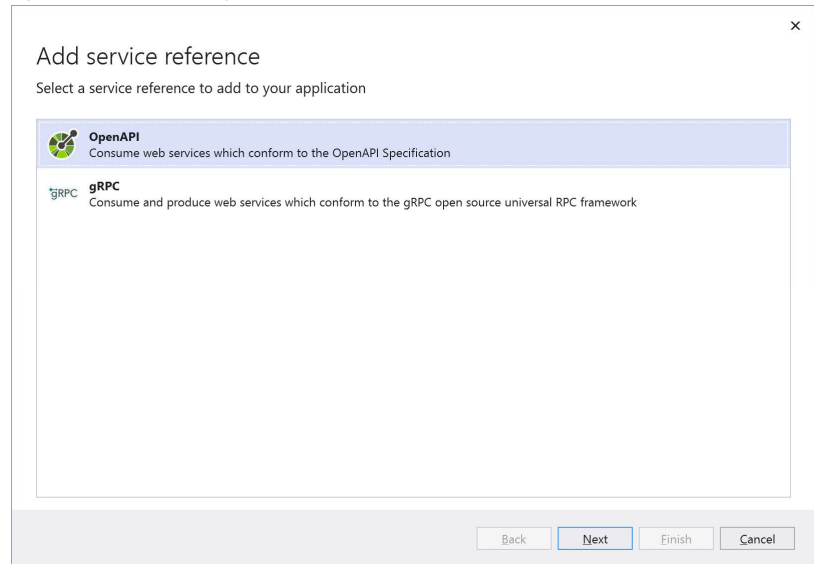
SwaggerClient Generierung

- Für eine SwaggerClient Generierung müssen wir *OpenAPI* auswählen



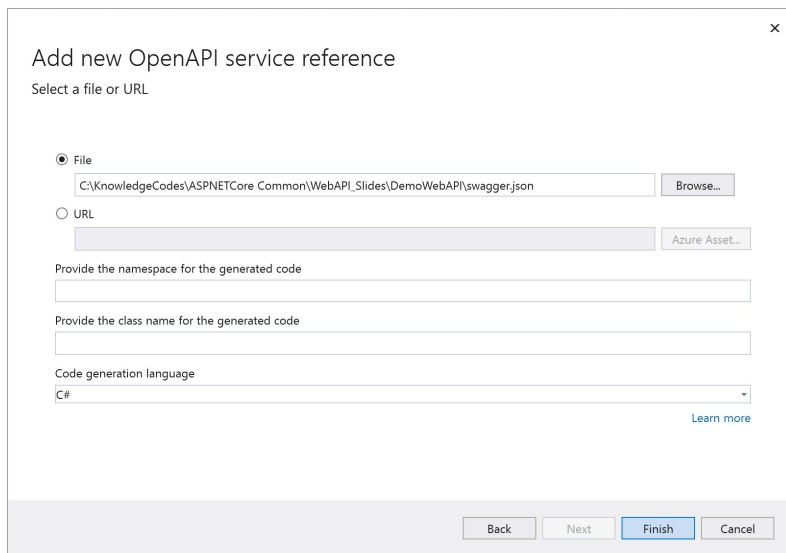
SwaggerClient Generierung

- Für eine SwaggerClient Generierung müssen wir *OpenAPI* auswählen



SwaggerClient Generierung

- Für eine SwaggerClient Generierung müssen wir *OpenAPI* auswählen



Add new OpenAPI service reference

Select a file or URL

☒ File

C:\KnowledgeCodes\ASPNETCore Common\WebAPI_Slides\DemoWebAPI\swagger.json [Browse...](#)

☐ URL

[Azure Asset...](#)

Provide the namespace for the generated code

Provide the class name for the generated code

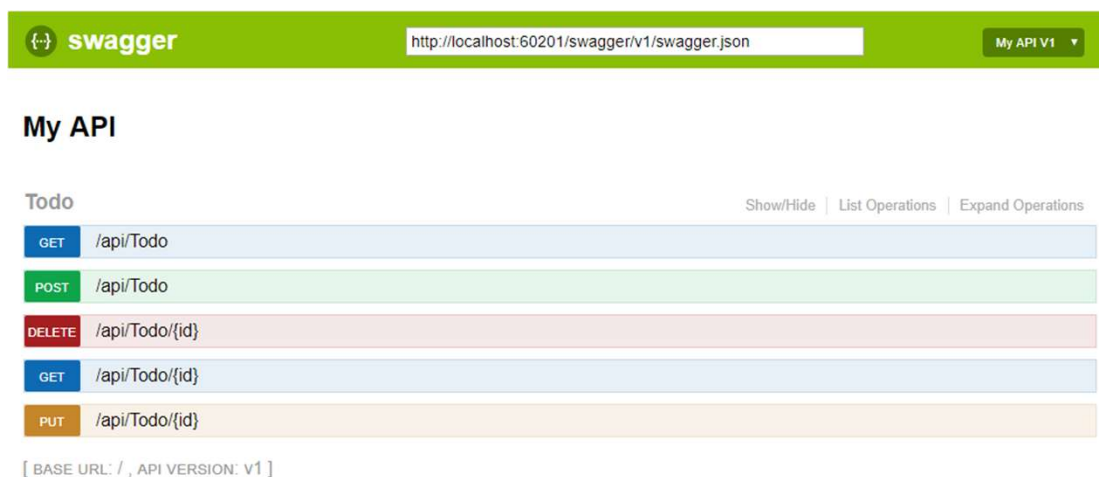
Code generation language

C# [Learn more](#)

[Back](#) [Next](#) [Finish](#) [Cancel](#)

SwaggerUI

- Für eine SwaggerClient Generierung müssen wir *OpenAPI* auswählen



SwaggerUI

- Für eine SwaggerClient Generierung müssen wir *OpenAPI* auswählen

Todo

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#)

GET /api/ToDo

Response Class (Status 200)

Success

Model Example Value

```
[
  {
    "id": 0,
    "name": "string",
    "isComplete": false
  }
]
```

Response Content Type | text/plain

Try it out!

[Hide Response](#)

Curl

```
curl -X GET --header 'Accept: application/json' 'http://localhost:60201/api/ToDo'
```

Request URL

```
http://localhost:60201/api/ToDo
```

Response Body

```
[
  {
    "id": 1,
    "name": "Item1",
    "isComplete": false
  }
]
```

Response Code

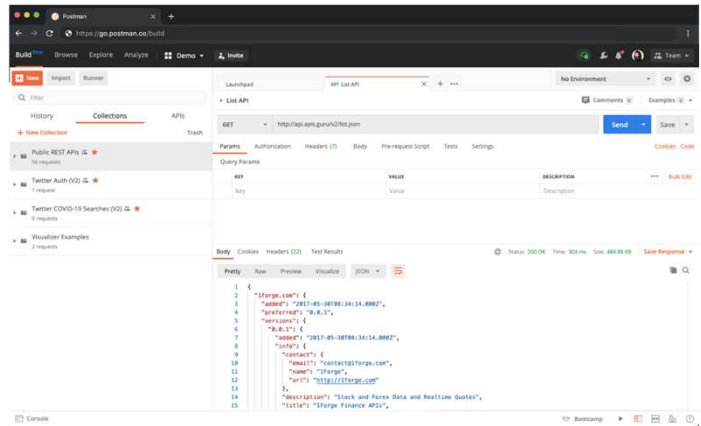
200

Response Headers

```
{
  "date": "Thu, 31 Aug 2017 17:29:04 GMT",
  "Server": "Kestrel",
  "transfer-encoding": "chunked",
  "Content-type": "Application/json; charset=utf-8"
}
```

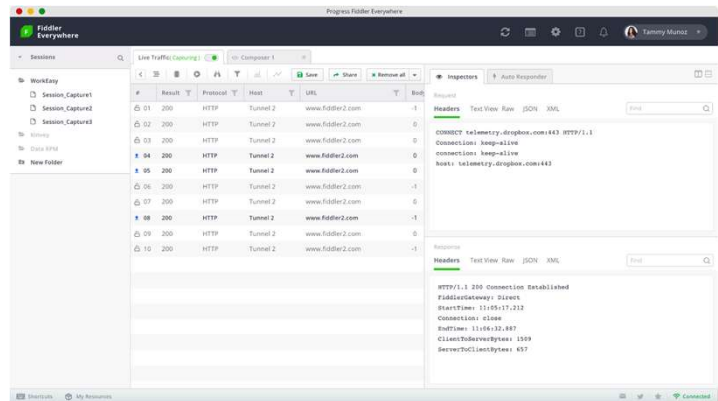
Weitere Tools zum Testen einer Web API - Postman

- Download -> [Postman | The Collaboration Platform for API Development](https://go.postman.co/build)



Weitere Tools zum Testen einer Web API - Fiddler

- Download -> [Fiddler - Web Debugging Proxy](#)
- [Telerik](#)



HTTP Statuscodes

Erweiterte Techniken

Die Bedeutung von HTTP - Status Codes

LEVEL 200 SUCCESS

200 – OK
201 – Created
204 – No Content

LEVEL 400 CLIENT ERROR

400 – Bad Request
401 – Unauthorized
403 – Forbidden
404 – Not Found
409 - Conflict

LEVEL 500 SERVER ERROR

500 – Internal
Server Error

21

IN-COURSE REMARKS

- In vorigen Beispielen wurde davon ausgegangen, dass alles gefunden wurde
- Wird nichts gefunden, sollte ein entsprechender Statuscode zurückgegeben werden

PREPARATION REMARKS

g
g

Die Bedeutung von HTTP - Status Codes - Details

Code	Nachricht	Bedeutung
200	OK	Erfolg, das Ergebnis der Anfrage wird in der Antwort übertragen.
201	Created	Erfolgreich bearbeitet, die Ressource wurde vor dem Senden der Antwort erstellt.
202	Accepted	Anfrage akzeptiert, wird aber erst später ausgeführt. Kein Gelingen garantiert
301	Moved Permanently	Redirect, die angeforderte Ressource steht unter einer anderen Adresse bereit. Die alte Adresse ist nicht länger gültig.
304	Not Modified	Mit Web Api Caching
400	Bad Request	Fehlerhafte Anfrage
401	Unauthorized	Keine gültige Authentifizierung, Anfrage nicht möglich.
402	Payment Required	Status für zukünftige HTTP-Protokolle reserviert – „ <i>Bezahlung benötigt</i> “. Wird oft verwendet um Fehler im Validierungsprozess aufzudecken.
403	Forbidden	Client nicht berechtigt, Anfrage wurde nicht durchgeführt.
404	Not Found	Die angeforderte Ressource wurde nicht gefunden. Anfrage ohne Grund abgewiesen.
409	Conflict	Anfrage wurde unter falschen Annahmen gestellt.
500	Internal Server Error	Unerwarteter Serverfehler.

IN-COURSE REMARKS

- In vorigen Beispielen wurde davon ausgegangen, dass alles gefunden wurde
- Wird nichts gefunden, sollte ein entsprechender Statuscode zurückgegeben werden

[HTTP-Statuscode – Wikipedia](#)

Web API und HTTP

- **Rückgabe HTTP Status Codes**
- HTTP *ActionResult* seit Version 2 (.NET 4.8) als Rückgabetyt möglich.
- In ASP.NET Core wird *ActionResult* verwendet.
- Anpassen des Controllers
 - Wenn GET Ressource nicht gefunden wurde, HTTP **404 Not Found** als Rückgabecode
 - POST, PUT, DELETE mit Rückgabetyt *ActionResult* anpassen
- Prüfung des Formats mit HTTP Debugger

Demo

23

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

??? HTTP Debugger ? Postman? Fiddler?

g

g

Web API Erweiterte Techniken

• Binding

- Aufruf einer Controller Methode mit Übergabeparametern durch die Web API
- HttpResponseMessage Put(int id, Product item) { ... }
- Id = primitive Type
- Product = Complex Type

- Complex Type aus URI: `[FromUri]`
- Complex Type aus Body: `[FromBody]`

24

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Web API und HTTP

• Content Negotiation

- „Inhaltsvereinbarung“ zwischen Client-Server
- XML oder JSON im **Accept** header
- Wird ein Format nicht gefunden: **406 Not Acceptable**
- XML basiert auf der ASP.NET Core Klassen **XmlSerializerInputFormatter** / **XmlSerializerOutputFormatter**
- **Accept Header**
 - Accept: application/xml
 - Accept: application/json; charset=utf-8
- Für POST und PUT „Content-Type“ statt „Accept“ Attribut

25

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g